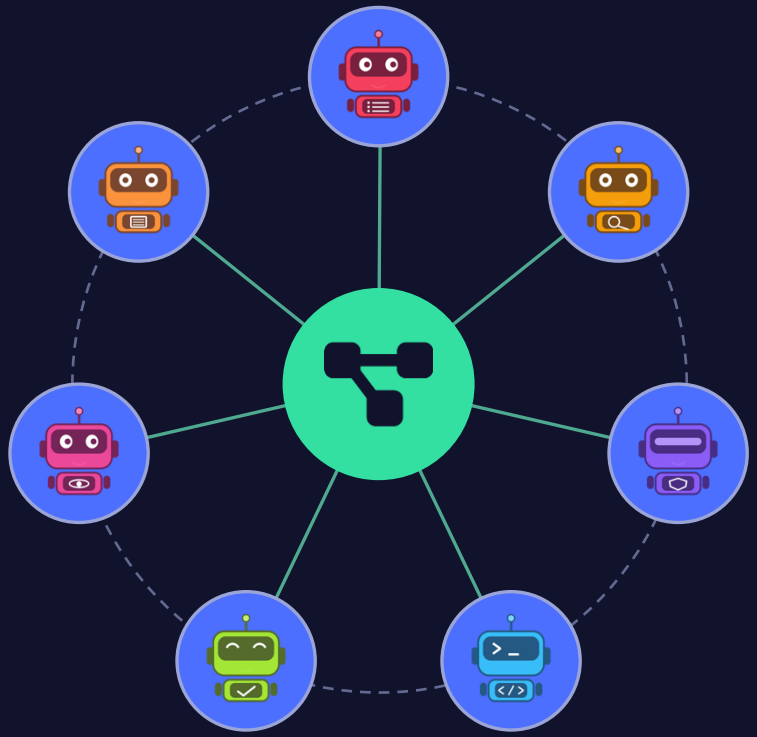
CS691 CAPSTONE · SPRINT 1 REVIEW

From One Brief to the Right Team

Sprint 1 turns a registry of AI agents into a team that assembles itself from a single plain-language brief.



AgentMesh · Team 2 Phoenix
Fall 2026

AGENDA

What we'll cover

1. **Team roles and responsibilities**
   Who owns what
2. **Improvements from feedback**
   What we changed since Sprint 0
3. **Project description**
   What AgentMesh is
4. **Team working agreement**
   How we work together
5. **Personas and MVP**
   Who it is for, what they can use
6. **Technologies and algorithms**
   The stack and the logic
7. **Diagrams**
   Architecture, context, ER, sequence, state, class
8. **Product and Sprint 1 backlog**
   Stories, criteria, points, tasks
9. **Test cases**
   23 cases across US_01 to US_05
10. **Metrics**
    Velocity, burndown, completed / committed
11. **Retrospective and Sprint 2**
    What we learned, what is next
12. **Project demo**
    Screenshots, API, GitHub, live demo

REVISED · FEEDBACK

Team roles and responsibilities



Dhwani Dobariya

Team Leader / Frontend Engineer

Leads planning and reviews; builds the /office UI



Om Jadhav

Full Stack Engineer

Builds features end to end across UI and API



Soumen Nageshkar

Full-Stack / Platform Engineer

Owns the platform and the orchestration core



Aniruddha Rath

Frontend Engineer

Builds frontend pages and components



Puneet Tulsiani

Backend Engineer

Builds the API, enrollment and data layer



Sutej Kulkarni

A.I. Engineer

Capability analysis, LLM providers and routing



Gurjot Singh

QA / Scrum Master

Runs Scrum events; owns test cases and QA

SPRINT 0 FEEDBACK

What we changed from feedback

Feedback	What we improved	Slide
Minimal personas	Full personas: photo, age, role, scenario, three goals and three pain points	7
Reading instead of presenting	Less text on each slide; talking points moved into speaker notes	All
Team photos	Professional headshots for every member, with role and responsibility	3
Bad mic / speaker	Mic and speakers tested together as a group before every recording	Recording
Working agreement is vague	Exact meeting days, times and place, plus clear rules for missed work	6
Retro was the aftermath	Sprint 1 retro held from scratch; each action item fixes one improvement item	29
Lazy screenshots (wiki)	Screenshots cropped to the feature being shown, no extra noise	22, 31

REVISED · FEEDBACK *marks every slide that changed because of this feedback*

PROJECT DESCRIPTION

What AgentMesh is

AgentMesh is an AI agent orchestration platform. It coordinates multiple specialised AI agents and multiple LLM providers to solve complex tasks, acting as an intelligent workflow engine for AI rather than a single chatbot.

01

Receive

A user submits one natural-language brief describing a complex task.

02

Decompose

The orchestrator analyses the brief and breaks it into the capabilities it needs.

03

Select and route

Only the relevant agents are spawned, in the right order, each on a suitable model.

04

Synthesise

Validated agent outputs are combined into one explainable result.

TEAM WORKING AGREEMENT

REVISED · FEEDBACK

How we work together

When and where

Stand-ups twice a week: Wednesday and Friday at 8 pm ET on google meet. Async updates daily in the team channel.

Commitment

If you can't finish a task, flag it in the channel at least 24 hours before the deadline so it can be re-assigned.

Deadlines

Individual work is due 48 hours before a Sprint review. No same-day drops on one person.

Shared load

Reviews and integration are shared across the team, not owned by a single 'fixer'.

Reminders

Teammates give one friendly reminder before a deadline; after that, it is on the individual.

Conflict

Disagreements are raised in the team meeting, not worked around silently. Unresolved issues go to the Team Leader.

PRODUCT PERSONAS

REVISED · FEEDBACK

Three people, three ways of working



Priya, 34

SOFTWARE ENGINEERING LEAD

Wants one orchestrated pipeline that runs security review, code fixes and tests before every merge.

GOALS

- One pipeline, end to end
- Security review before merge
- Automated fixes and tests

PAIN POINTS

- Manually chaining AI tools
- Slow, error-prone handoffs
- Fragmented tool results



Daniel, 29

DATA ANALYST

Wants cleaning, analysis and reporting from a single plain-language request.

GOALS

- One plain-language request
- Cleaning, analysis, reporting
- Clean structured data handoff

PAIN POINTS

- Models lose track of steps
- Messy stage-to-stage handoffs
- Re-prompting at every stage



Aisha, 31

TECHNICAL PRODUCT MANAGER

Wants to see which capability ran, why it was chosen and what it cost, for every workflow.

GOALS

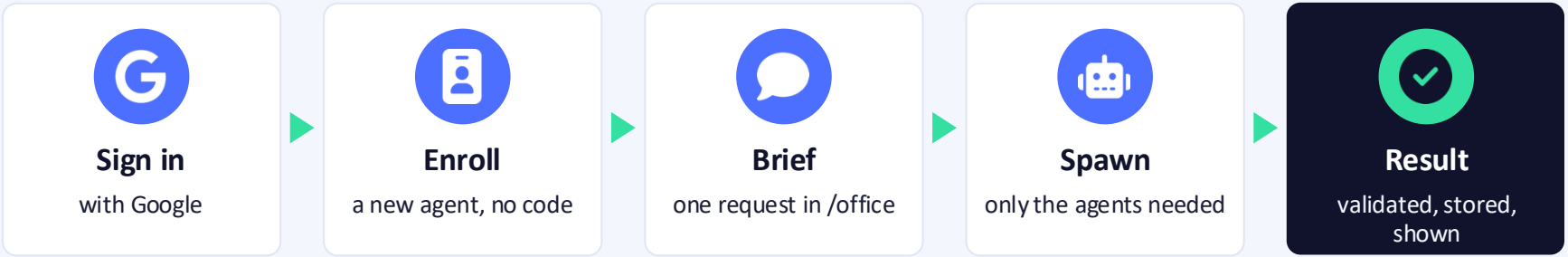
- See which capability ran, and why
- Cost visibility per workflow
- Audit latency, decision quality

PAIN POINTS

- Black-box chatbot answers
- No cost audit trail
- Latency and quality unclear

MINIMAL VIABLE PRODUCT

What a user can actually do today



IN THE MVP

- Agent enrollment with a schema-validated results table per agent
- Capability-based spawning with a reason for every pick and skip
- Output validation before anything is stored or passed on
- The /office workspace to brief, browse agents and watch progress

WHY IT IS AN MVP

A user goes from one plain-language brief to a coordinated, validated result without calling the API or writing code. That is the smallest version of AgentMesh that is useful on its own.

TECHNOLOGIES

The stack behind AgentMesh

FRONTEND

- Next.js
- React
- TypeScript

BACKEND

- Python
- FastAPI
- SQLAlchemy
- Pydantic

DATA

- PostgreSQL
- Redis 8

AI PROVIDERS

- OpenAI
- Anthropic
- Gemini
- Hugging Face

DEVOPS AND MONITORING

- Docker
- GitHub Actions
- Prometheus
- Grafana

AUTH AND QUALITY

- Google
- JWT
- pytest
- Playwright

ALGORITHMS

The logic behind the orchestrator

Capability classification

Heuristic or LLM analysis maps a brief to the capabilities it needs, with a clarity check for vague briefs.

Minimal covering set

The selector spawns the fewest enabled agents that cover every capability and records why each other agent was skipped.

DAG planning

Topological ordering builds sequential, parallel and conditional steps; plans with cycles are rejected.

Conditional branching

Edge conditions read intermediate results, for example skipping the coding step when no vulnerability is found.

Cost-aware routing

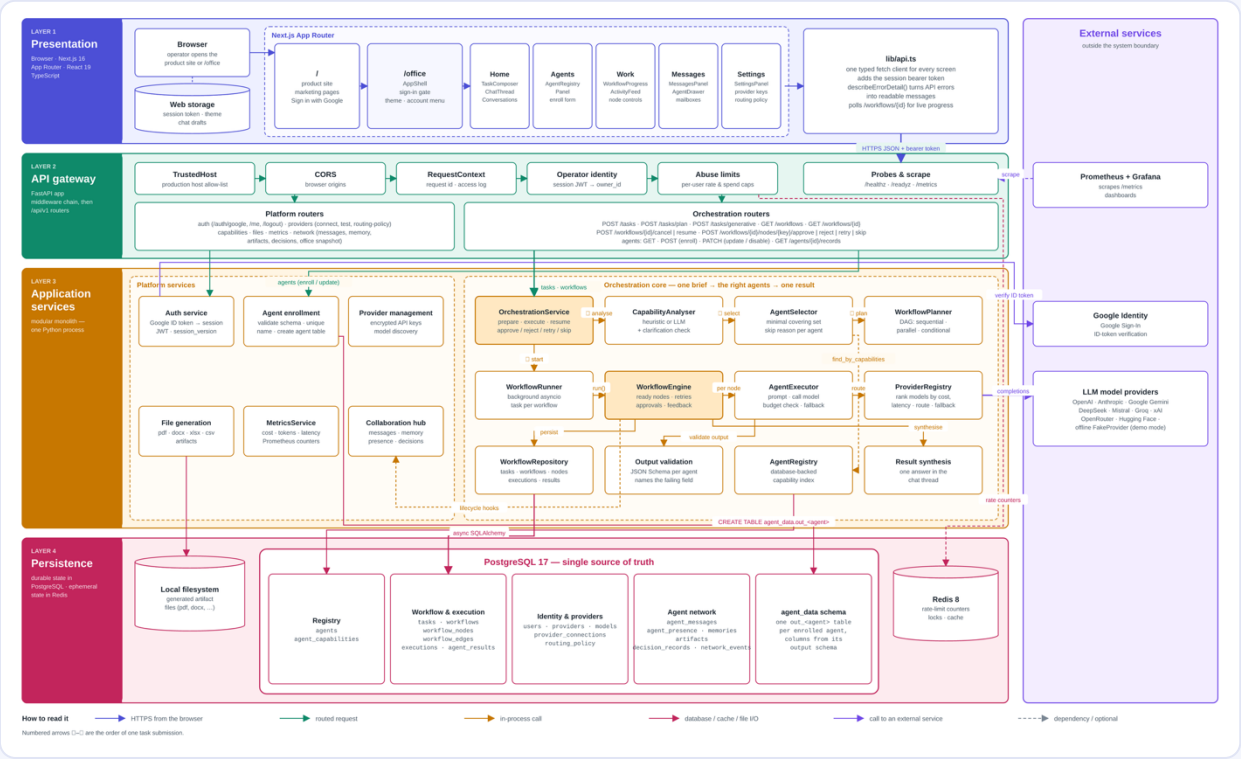
Models are ranked by cost and latency per step, with logged fallback to the next provider.

Retry and schema validation

Each output is checked against JSON Schema; invalid output is retried with backoff up to three times.

ARCHITECTURE DIAGRAM

Four layers, one request path

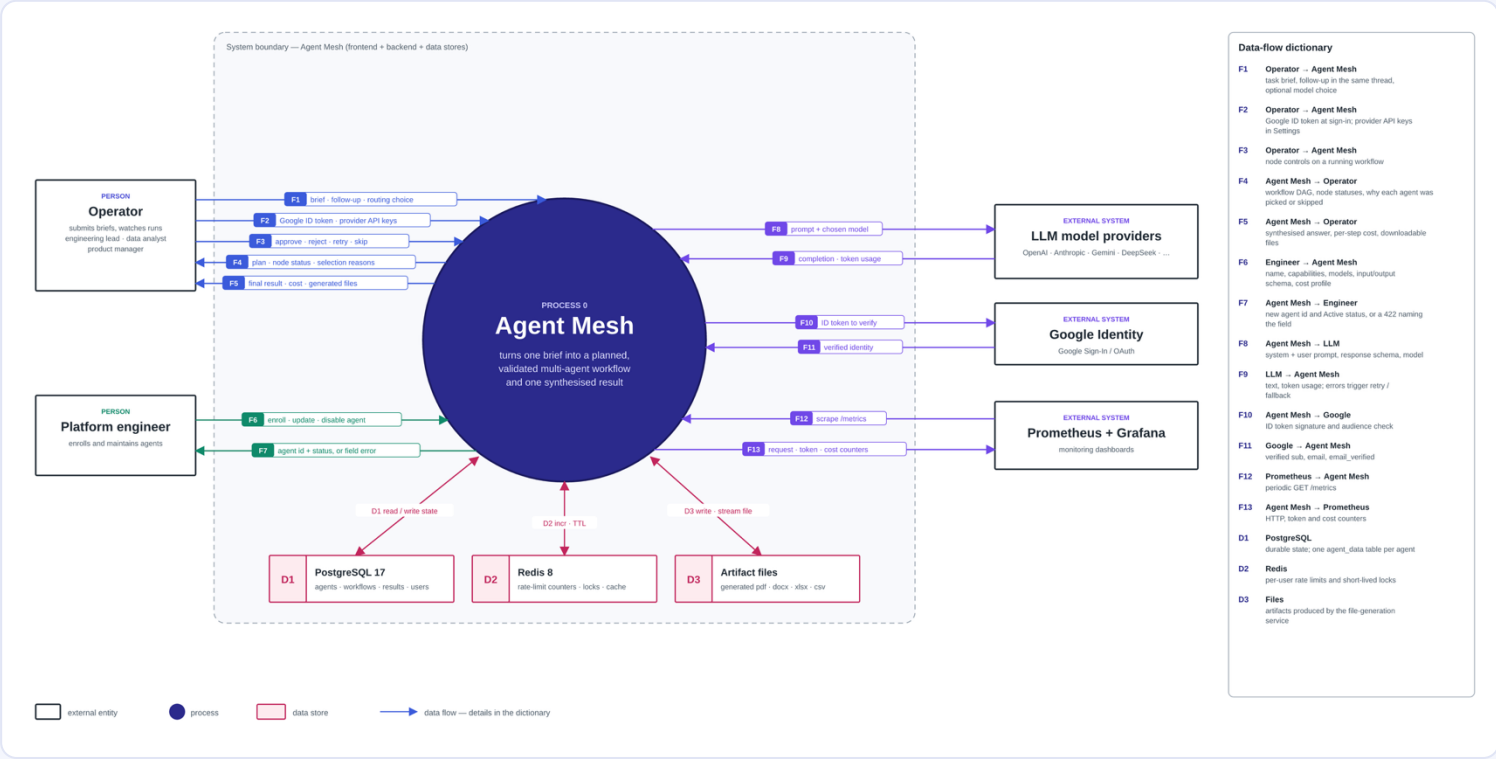


HOW TO READ IT

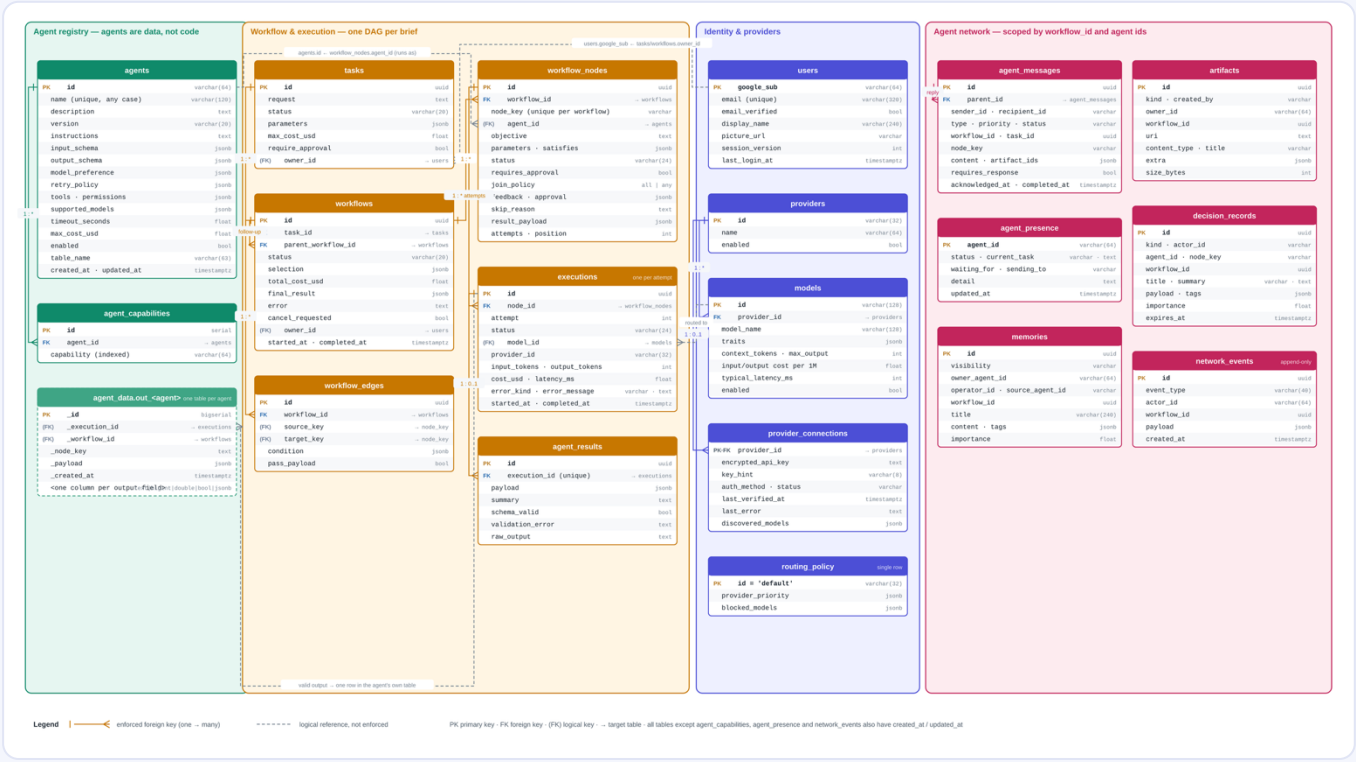
- Presentation: Next.js /office pages and one typed API client
- API gateway: FastAPI middleware chain, then platform and orchestration routers
- Application services: the orchestration core picks, plans and runs agents
- Persistence: PostgreSQL 17 as the source of truth, Redis 8 for counters and locks
- External: LLM providers, Google Identity, Prometheus + Grafana

CONTEXT DIAGRAM

Agent Mesh and the world around it

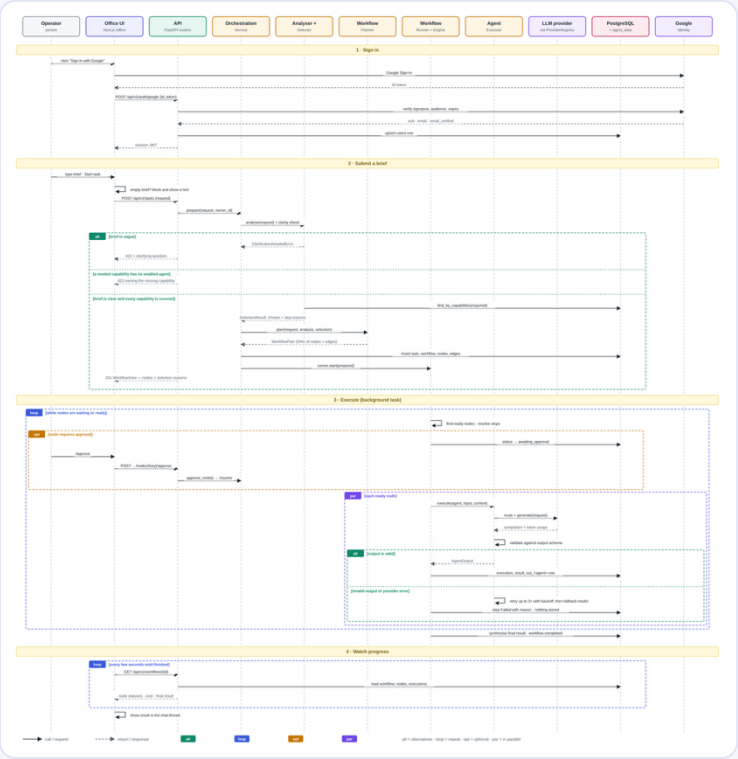


The database behind the agents



SEQUENCE DIAGRAM

Sign in, submit a brief, run it

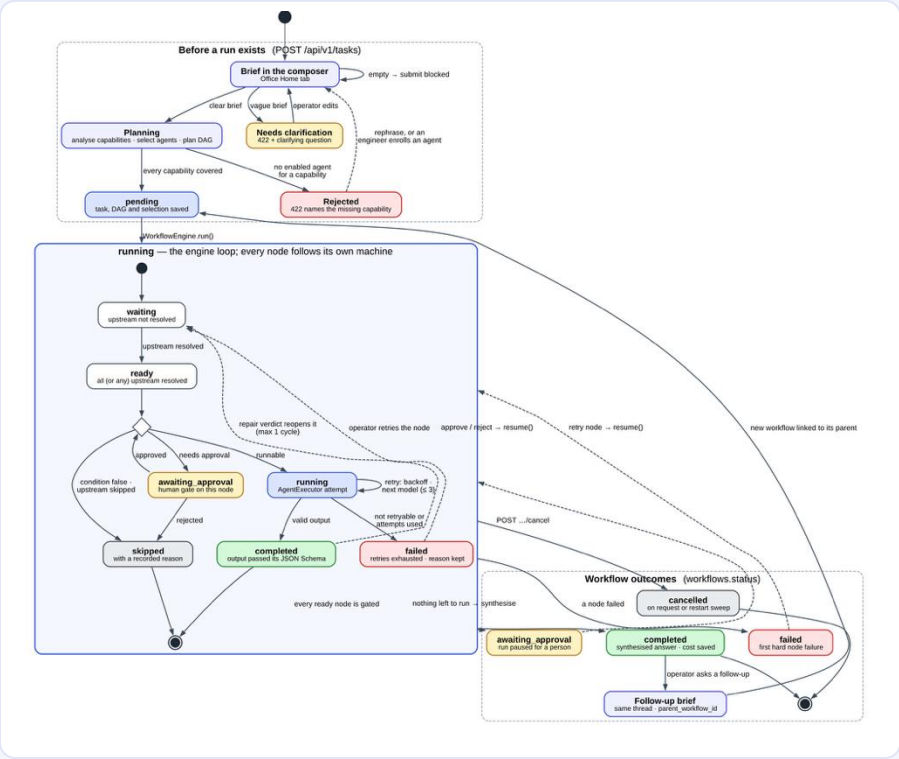


HOW TO READ IT

- 1 Sign in: Google ID token is verified and a session JWT is issued
- 2 Submit a brief: vague briefs get a clarifying question, missing capabilities get a 422, otherwise the plan is saved
- 3 Execute: ready nodes run in parallel; output is validated, retried, or the step fails with a reason
- 4 Watch progress: the UI polls the workflow until it finishes

STATE DIAGRAM

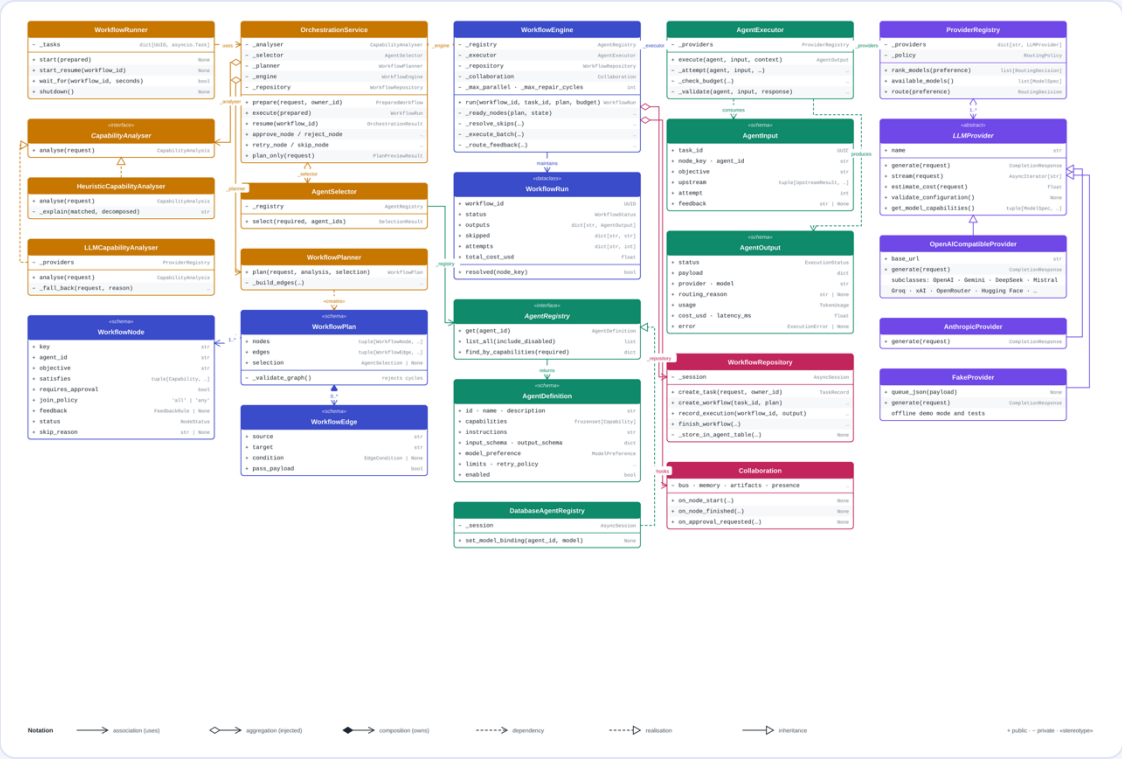
Every brief and node has a state



HOW TO READ IT

- Before a run: a brief is planned, sent back for clarification, or rejected with the missing capability
- While running: each node moves waiting → ready → running, or pauses for approval or is skipped
- A node ends completed (valid output) or failed (retries used up)
- The workflow ends completed, failed, cancelled, or awaiting approval; a follow-up starts a new linked workflow

How the backend classes fit together



HOW TO READ IT

- OrchestrationService wires the analyser, selector, planner and engine
- WorkflowEngine runs ready nodes through AgentExecutor
- AgentRegistry is an interface; DatabaseAgentRegistry is the app implementation
- LLMProvider is abstract: OpenAI-compatible, Anthropic and Fake providers plug in

SPRINT 1 BACKLOG

Five user stories committed, 42 points

ID	Feature	User story	Pts	Criteria	Status
US_01	**Agent Enrollment**	Platform engineer: enroll a new agent through one workflow so the orchestrator can use it without code changes	**8**	5 / 5	ACCEPTED
US_02	**Agent Spawning**	User: the system reads my brief and spawns the right agents, more than one when needed	**13**	5 / 5	ACCEPTED
US_03	**Dynamic Table Creation**	Data analyst: a table is created automatically from each agent's output schema	**8**	4 / 4	ACCEPTED
US_04	**Schema Validation**	User: schemas and outputs are validated so bad data never reaches storage or the next step	**5**	4 / 4	ACCEPTED
US_05	**Frontend**	User: one workspace to submit a brief, see enrolled agents and watch spawned agents	**8**	4 / 4	ACCEPTED
Total		**5 stories committed, 5 completed**	**42**	**22 / 22**	

US_01 · AGENT ENROLLMENT · 8 PTS

Enroll agents, not code

> *As a platform engineer, I want to enroll a new agent through one defined workflow so the orchestrator can discover and use it without code changes.*

ACCEPTANCE CRITERIA

ACCEPTED

- Agent record holds name, description, capabilities, models, schemas, cost ceiling
- Registered agent gets a unique ID and an Active (enabled) status
- Duplicate IDs and missing required fields are rejected with a clear error
- Enrollment completes in a single request (POST /agents) in under 3 seconds
- Agents can be updated or disabled; disabled agents are never selected

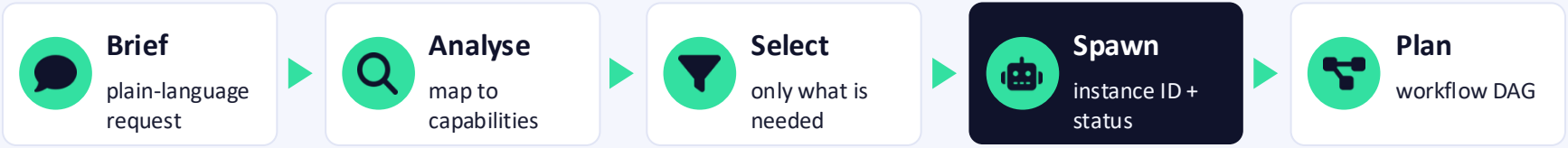
How we built it

- Registry protocol with in-memory (tests) and database (app) implementations
- Capabilities are their own indexed table, so lookup is a join, not a JSON scan
- POST /agents enrolls; PATCH updates or disables, and disabled agents are never offered
- Proof test: a brand-new agent is registered and selected with zero engine changes

SCRUM-10 · SCRUM-11 · SCRUM-12

US_02 · AGENT SPAWNING · 13 PTS

From brief to the right team



NARROW BRIEF

"Find security vulnerabilities in the payments service"

security-agent

1 agent spawned
6 skipped, each with a recorded reason

BROAD BRIEF

"Analyze this repo for vulnerabilities, fix the issues, write tests, and document the changes"

planning · security · coding · testing · documentation

5 agents spawned in order
research and review skipped

CRITERIA

ACCEPTED

- Brief mapped to capabilities; one or more agents spawned
- Unneeded agents skipped with a reason
- Each spawned agent has its own node ID and status
- No match: nothing spawned, 422 names the missing capability
- Empty briefs blocked; vague briefs get a clarifying question

US_03 · DYNAMIC TABLE CREATION · 8 PTS

A results table for every agent

> *As a data analyst, I want a table created automatically for each agent from its output schema so results are stored without manual database changes.*

ACCEPTANCE CRITERIA

ACCEPTED

- A table is created at enrollment, columns and types from the output schema
- Table names generated safely and uniquely; unsafe names or SQL rejected
- Running creation twice gives no error and no duplicate table
- If table creation fails, enrollment rolls back with nothing half-created

How we built it

- Enrollment derives a safe table name and stores it on the agent record
- agent_data.out_<agent>: one table per agent, one typed column per output field
- CREATE TABLE runs inside the enrollment transaction, so a failure rolls both back
- Each valid output is written as one row in that agent's own table

SCRUM-10 · SCRUM-12

US_04 · SCHEMA VALIDATION · 5 PTS

Bad data never reaches storage

> *As a user, I want agent schemas and outputs validated so that bad data never reaches storage or the next step.*

ACCEPTANCE CRITERIA

ACCEPTED

- Malformed schema rejected at enrollment with an error naming the field
- Output validated against the agent's JSON Schema before it is written
- Missing field or wrong type: step marked failed, nothing stored
- Validation errors logged with agent ID and field path

How it works

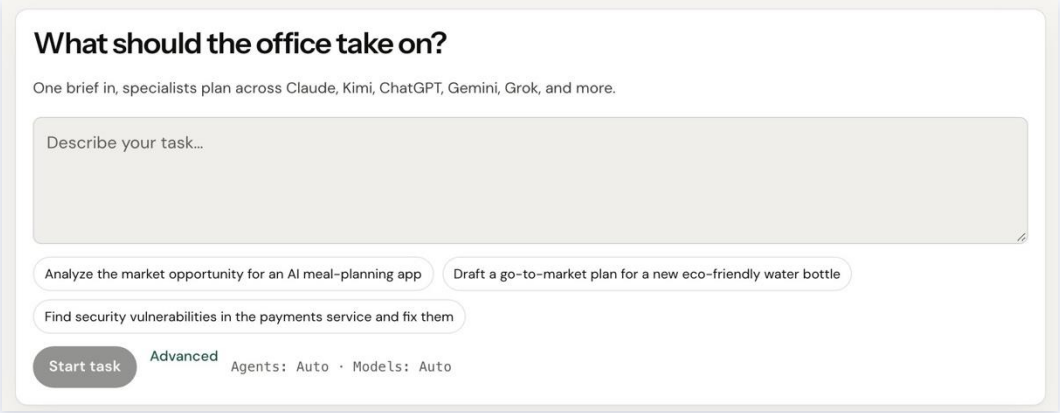
- LLM output is treated as untrusted and parsed against JSON Schema (Draft 2020-12)
- Errors name the field path, for example findings/0/severity
- Invalid output is a retryable failure; raw text kept for debugging
- Workflow plans are validated as DAGs: no cycles, valid edges

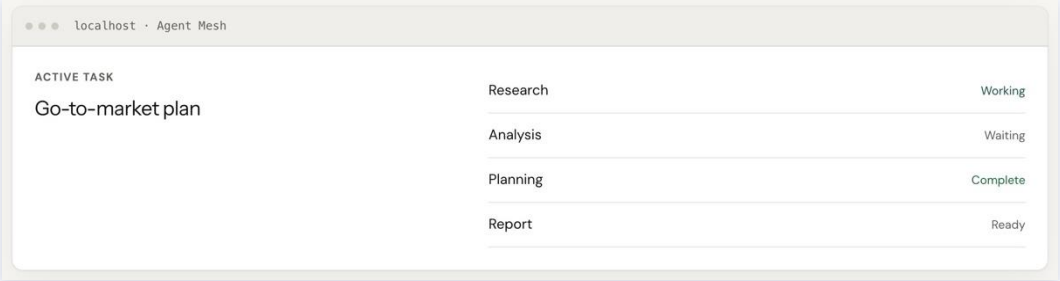
SCRUM-9 · SCRUM-11

US_05 · FRONTEND · 8 PTS

REVISED · FEEDBACK

One workspace, no API calls needed





ACCEPTANCE CRITERIA

ACCEPTED

- /office has a brief box and Start task; empty briefs keep the button disabled
- Agents page lists enrolled agents with capabilities and status from the API
- After submit, spawned agents appear with status and selection reason
- Backend validation and error messages are shown clearly

Nine tasks that delivered the stories

Ship it

SCRUM-5 → All

Project root setup

.gitignore, .env.example, Makefile, README

SCRUM-6 → US_05

Docker Compose environments

4 services, health-gated, hot reload

SCRUM-7 → All

GitHub Actions CI

5 automated jobs on every PR

Backend core

SCRUM-8 → All

Backend requirements, pyproject

pinned deps, ruff, mypy, pytest

SCRUM-9 → US_02 · US_04

Config, logging, middleware

typed settings, structlog, request IDs

SCRUM-10 → US_01 · US_03

Database session handling

async engine, sessions, base classes

Data and design

SCRUM-11 → US_01 · US_04

SQLAlchemy models + schemas

10 tables, Pydantic contracts

SCRUM-12 → US_01 · US_03

Alembic migrations

versioned, reversible schema

SCRUM-13 → US_02

Architecture documentation

architecture.md and ADRs

23 test cases, all passing

TC	Story	Scenario	Type	Result
TC_01	US_01	Enroll an agent with valid details	Positive	Pass
TC_02	US_01	Enroll with a missing required field	Negative	Pass
TC_03	US_01	Enroll a duplicate agent name	Negative	Pass
TC_04	US_01	Single fast enrollment request	Positive	Pass
TC_05	US_01	Disable an enrolled agent	Positive	Pass
TC_06	US_02	Brief needing several capabilities	Positive	Pass
TC_07	US_02	Brief needing one capability	Positive	Pass
TC_08	US_02	Brief with no matching agent	Negative	Pass
TC_09	US_02	Empty or vague brief	Negative	Pass
TC_10	US_02	Same brief submitted twice	Positive	Pass
TC_11	US_02	Selection reason is shown	Positive	Pass
TC_12	US_03	Table created on enrollment	Positive	Pass

TC	Story	Scenario	Type	Result
TC_13	US_03	Unsafe table name	Negative	Pass
TC_14	US_03	Repeat table creation	Positive	Pass
TC_15	US_03	Table creation failure rolls back	Negative	Pass
TC_16	US_04	Reject a malformed schema	Negative	Pass
TC_17	US_04	Store valid agent output	Positive	Pass
TC_18	US_04	Output missing a required field	Negative	Pass
TC_19	US_04	Output with the wrong data type	Negative	Pass
TC_20	US_05	Submit a brief from the UI	Positive	Pass
TC_21	US_05	Empty brief blocked in the UI	Negative	Pass
TC_22	US_05	Agents page matches the API	Positive	Pass
TC_23	US_05	Backend error shown to the user	Negative	Pass

23 / 23 passed 12 positive · 11 negative · automated in CI on Oct 18, 2026

SPRINT 1 OUTCOME

Stories completed and not completed

COMPLETED

5 stories · 42 pts

US_01	**Agent Enrollment**	8 pts
US_02	**Agent Spawning**	13 pts
US_03	**Dynamic Table Creation**	8 pts
US_04	**Schema Validation**	5 pts
US_05	**Frontend**	8 pts

NOT COMPLETED

0

stories carried over

Every committed story met all of its acceptance criteria, so nothing moves to Sprint 2.

SPRINT 2 AND SPRINT 3

Product backlog

Sprint 2 - Workflow Engine and Agent Execution

End of sprint: the agents spawned in Sprint 1 are connected into a workflow graph that actually runs, with sequential and parallel steps, retries, live progress and real specialist agents doing the work.

US_ID	Feature	User Story	Acceptance Criteria	Story Points
US_06	**Workflow Planning**	As a user, I want my spawned agents arranged into a workflow plan so that they run in the right order with the right dependencies.	- A workflow DAG is generated with agents as nodes and dependencies as edges. - The plan supports sequential steps, parallel branches with joins, and conditional steps. - Circular dependencies are detected and rejected before execution. - The plan is saved and can be viewed in the workspace.	**13**
US_07	**Workflow Execution**	As a user, I want the workflow to run on its own with live progress so that complex jobs finish without manual prompting and I can see where they stand.	- Independent steps run in parallel; dependent steps wait for their inputs. - Output from one agent is validated, stored in its table and passed to the next agent. - A failed step is retried up to 3 times with backoff, then marked Failed with a reason; completed results are kept. - Each step shows Pending, Running, Completed, Failed or Skipped and updates without a page refresh.	**13**
US_08	**Specialist Agents**	As a software engineering lead or data analyst, I want working specialist agents so that I can run a review-fix-test or clean-analyse-report pipeline from one request.	- Each agent has an input/output schema, a table and unit tests. - The code auditor returns issues by severity; the code fixer acts only on those issues; the test writer produces runnable tests. - The data cleaner returns a cleaned dataset with a change summary; the analyst and report writer turn it into a readable report.	**13**

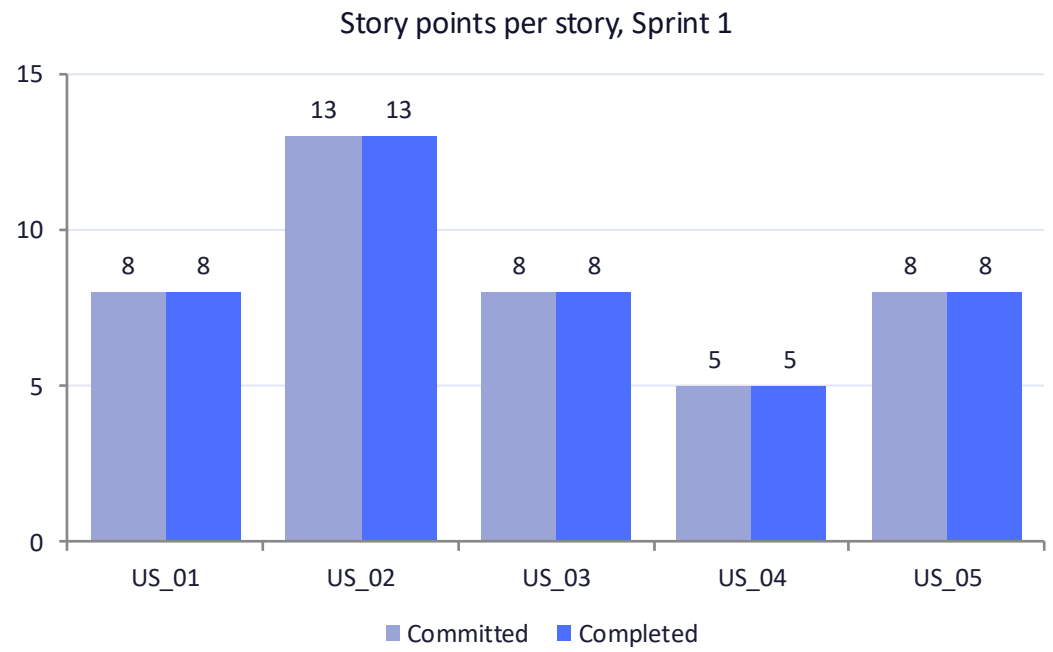
Product backlog

Sprint 3 - Multi-Provider Routing, Cost Transparency and Final Result

End of sprint: every step is routed to a suitable model with automatic fallback, the user can see what each workflow cost, and the output is delivered as one synthesised result in a chat thread.

US_ID	Feature	User Story	Acceptance Criteria	Story Points
US_09	**Model Routing**	As a user, I want each agent step routed to a suitable model across providers, with automatic fallback, so that workflows complete reliably without wasting spend.	- Adapters for OpenAI, Anthropic and Google Gemini sit behind one shared interface. - Routing considers task type, cost and latency, and the model used for each step is stored and shown. - If a provider errors or is rate-limited, the step switches to the fallback model within 5 seconds. - The user can see when a fallback was used.	**13**
US_10	**Cost Transparency**	As a technical product manager, I want cost, token usage and latency per agent and per workflow so that I can audit spend and performance.	- Tokens, estimated cost and latency are recorded for every step. - A workflow summary shows totals and a per-agent breakdown. - Cost data is saved so it can be reviewed later.	**5**
US_11	**Result Synthesis**	As a user, I want all validated agent outputs combined into one clear result in a chat thread so that I can read it and ask follow-ups.	- The final result appears in the chat thread and credits the agent behind each part. - The user can send a follow-up in the same thread and context is kept. - The result and conversation are saved and reload when the thread is reopened.	**13**

Velocity and completion ratio



TEAM VELOCITY

42

story points completed

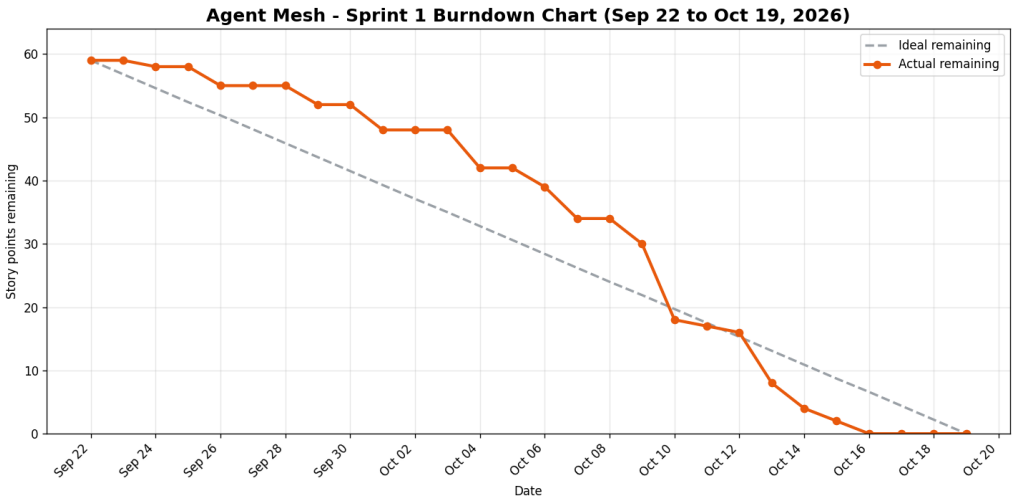
COMPLETED / COMMITTED

100%

42 of 42 points

METRICS · BURNDOWN CHART

Sprint 1 burndown



COMPLETED TASKS

9 / 9

engineering tickets closed

- ✓ SCRUM-5 Project root setup
- ✓ SCRUM-6 Docker Compose
- ✓ SCRUM-7 GitHub Actions CI
- ✓ SCRUM-8 Backend requirements
- ✓ SCRUM-9 Config and logging
- ✓ SCRUM-10 DB session handling
- ✓ SCRUM-11 Models and schemas
- ✓ SCRUM-12 Alembic migrations
- ✓ SCRUM-13 Architecture docs

RETROSPECTIVE

REVISED · FEEDBACK

What Sprint 1 taught us

What went well	What can be improved	Action items
• Completed Sprint 1 Planning effectively, establishing clear roles right from day one.	• The 13-point ticket for multi-agent spawning was too massive, creating a delivery bottleneck.	• Split any ticket larger than 5 story points into smaller, independent sub-tasks during planning.
• Mapped out the initial foundation for the core Agent Mesh Enrollment Workflow.	• Schema validation and dynamic table creation stalled because verifying structures required manual checks.	• Implement automated schema assertion tests to validate database fields on every code commit.
• Made significant theoretical headway on core optimization logic before code modification.	• Technical documentation and Wiki page updates were neglected early on, resulting in a last-minute rush.	• Enforce a strict 'Definition of Done' where a feature cannot be closed until its documentation is written.
• Kicked off frontend design and development tasks early to provide visual mock-ups.	• Technical dependencies between the frontend interface and backend workflows caused idle blocking time.	• Create mock API schemas on Day 1 of the sprint so frontend and backend teams can develop in parallel.
• Solidified user stories and technical acceptance criteria to establish product guardrails.	• End-to-end testing of workflows lacked a dedicated setup, resulting in local environment confusion.	• Build a lightweight Docker Compose script to launch the entire agent mesh locally with one command.
• Structured the initial metrics tracking framework to gauge agent processing efficiency later.	• Minor feature scope creep entered the sprint via loose parameters, delaying optimization code.	• Require explicit Product Owner approval before any functional scope adjustments are made mid-sprint.

SPRINT 2 · PLANNED AND COMMITTED

Stories committed for Sprint 2

US_06	**Workflow Planning** Agents arranged into a validated DAG, saved and viewable	13 pts
US_07	**Workflow Execution** Parallel steps, retries and live progress without refresh	13 pts
US_08	**Specialist Agents** Review-fix-test and clean-analyse-report pipelines	13 pts

COMMITTED

39 pts

CARRY-OVER FROM SPRINT 1

None

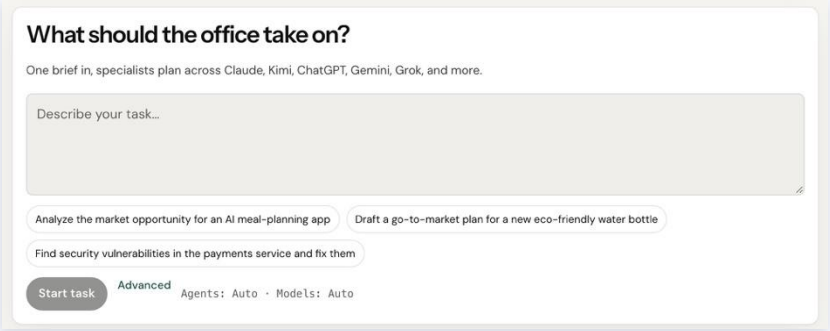
all five stories accepted



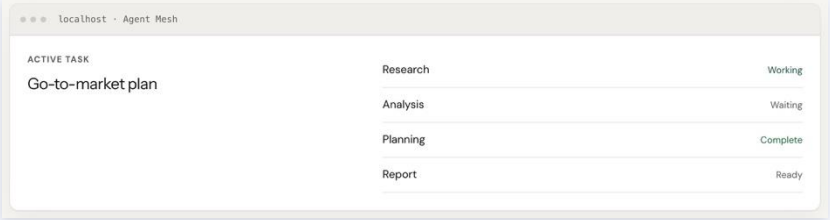
Sprint 2 goal the agents spawned in Sprint 1 run as a workflow graph with sequential and parallel steps, retries, live progress and real specialist agents.

REVISED · FEEDBACK

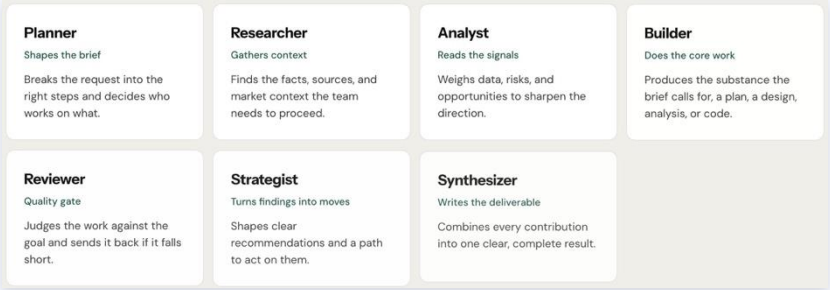
What a user sees, step by step



1 **Brief** type one request or pick a suggestion



3 **Live status** working, waiting, complete, ready



2 **Specialists** only the desks a brief needs light up

Built with

- Next.js App Router + TypeScript
- Light and dark themes with accessible contrast
- Typed API client shared by every page

The API behind every screen

POST	`/api/v1/auth/google`	Sign in with a Google ID token
POST	`/api/v1/agents`	Enroll an agent and create its table
PATCH	`/api/v1/agents/{id}`	Update or disable an agent
GET	`/api/v1/agents?capability=…`	List or filter enrolled agents
POST	`/api/v1/tasks/plan`	Preview which agents a brief spawns
POST	`/api/v1/tasks`	Submit a brief and run the workflow
GET	`/api/v1/workflows/{id}`	Node status, reasons, cost, result

EXAMPLE

```
GET /api/v1/agents
  ?capability=
  security.vulnerability_analysis

200 OK
["security-agent"]

POST /api/v1/tasks
  {"request": "help me"}
422  clarifying question
```

GITHUB

Source code and wiki

github.com/htmw/F2026-T2-Phoenix

`frontend/`	Next.js app and Playwright tests
`backend/`	FastAPI app, migrations and pytest suite
`docs/`	architecture, ADRs, agile artifacts, slides
`Wiki`	deliverables, diagrams, personas, test cases

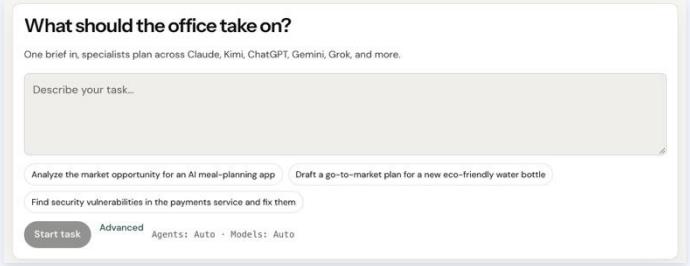
```
$ docker compose up --build
```

Scan to open the repository

LIVE APPLICATION DEMO

Let's run it

1. `docker compose up --build`
   log: agents_seeded created=7 total=7
2. `GET /api/v1/agents`
   all seven, with limits and retry policies
3. `?capability=security.vulnerability_analysis`
   filter: exactly ["security-agent"]
4. `GET /api/v1/capabilities`
   17 capabilities, none orphaned
5. `POST /api/v1/tasks/plan`
   agents selected and skipped, each with a reason
6. `open localhost:3000/office`
   submit a brief, watch agents spawn



Recorded demo

https://agent-mesh-web.onrender.com/

THANK YOU

Questions?

github.com/htmw/F2026-T2-Phoenix